Build a Q&A API with Python & Django

Get set up while we wait. In a terminal:

```
git clone https://github.com/joshualent/questions-api.git
cd questions-api
uv sync --locked
uv run manage.py migrate
uv run manage.py runserver
```



github.com/
joshualent/
questions-api

Done? Open `http://127.0.0.1:8000/admin/`. You should see a login page.

No Python 3.14? `uv sync` installs it for you.

Placeholder: Developer Club logo (set logo in workshop.config.ts)

Build a Q&A API with Python & Django

Backend workshop · Developer Club, WMU

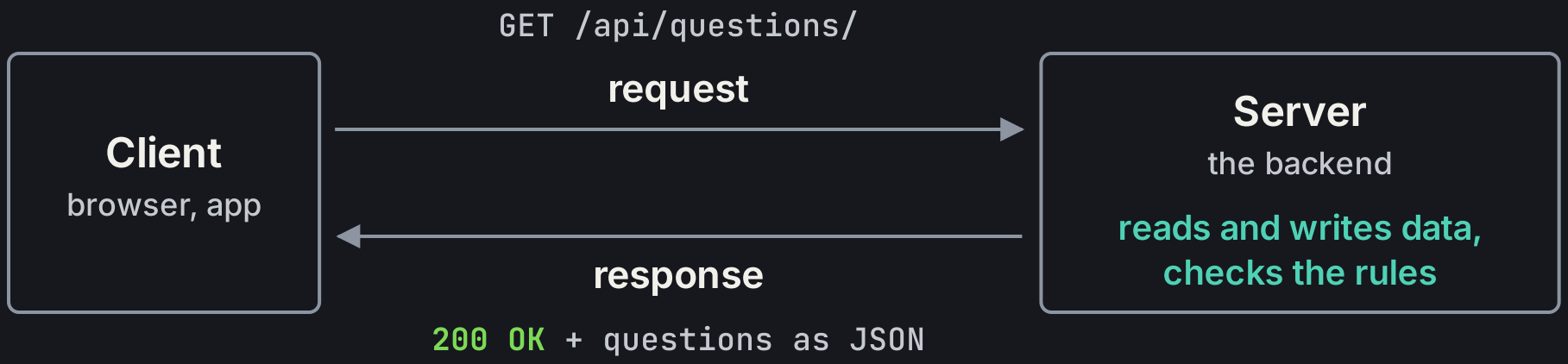
Monday, October 19, 2026 · Floyd Hall D-109

Tonight

We build an API where people ask questions, vote on them, and mark them answered.

- You code along on your own laptop.
- Every lesson ends on a checkpoint branch, so nobody stays stuck.
- Stuck anyway? Put a hand up. Helpers are walking around.

What a backend does

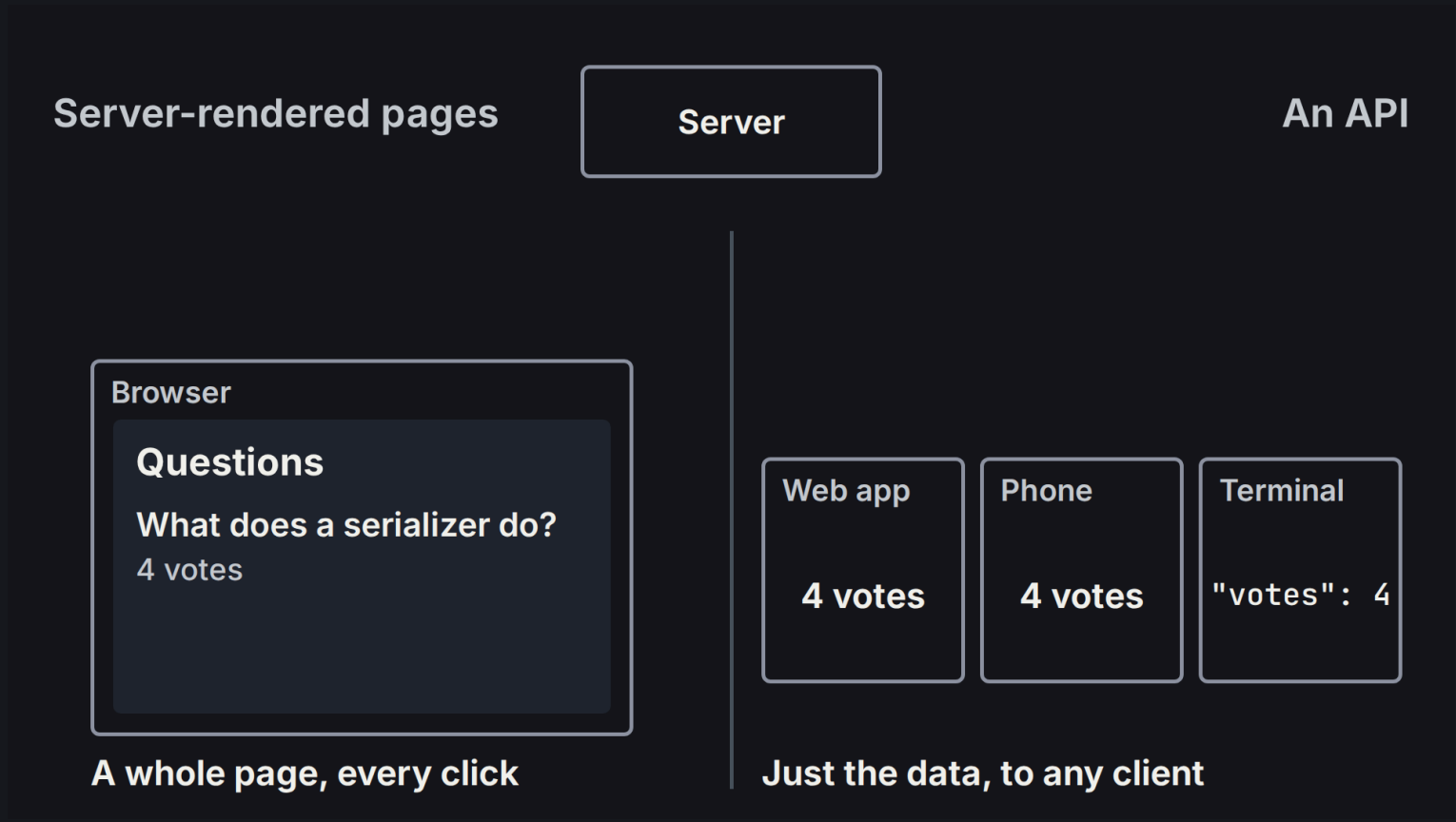


What the backend owns

- **Storing data**: questions, votes, users.
- **Enforcing rules**: a question can't be blank.
- **Deciding who can do what**: only moderators delete.

Never trust the client.

Server-rendered pages vs. an API



Why build an API?

- One backend serves many clients: a web app, a phone, a terminal.
- The frontend and backend can change separately.
- Each side can be tested and scaled on its own.

Server-rendered apps are still a fine choice for many projects.

Anatomy of a request

Request

```
POST /api/questions/ HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"text": "What does a serializer do?"}
```

Response

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 1,
 "text": "What does a serializer do?",
 "votes": 0,
 "answered": false,
 "created_at": "…"}
```

The server filled in id, votes, answered, and created_at.

Methods and status codes tonight

Methods			Status codes	
`GET`		read	`200 OK`	it worked
`POST`		create, or run an action	`201 Created`	created
`PUT`	`PATCH`	edit	`400 Bad Request`	blank text
`DELETE`		delete	`403 Forbidden`	not allowed
			`404 Not Found`	no such question

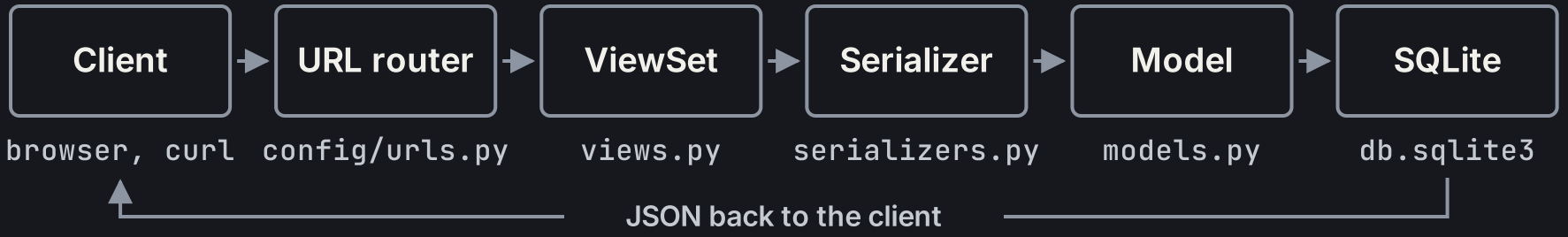
Resources and URLs

`/api/questions/` is the collection: every question.
`/api/questions/1/` is one question, with id 1.

`GET`	`/api/questions/`	list
`POST`	`/api/questions/`	create
`GET`	`/api/questions/1/`	read question 1
`PUT`	`/api/questions/1/`	replace it
`PATCH`	`/api/questions/1/`	update it
`DELETE`	`/api/questions/1/`	delete it

Later we add actions: `POST` `/api/questions/1/upvote/`

The map



We'll build it right to left: the model first.

Life of a POST



What went in, and what came back

Sent `{"text": "What does a serializer do?", "votes": 9999}`

Got back `201 Created` `{"id": 1, …, "votes": 0, …}`

Blank text `400 Bad Request` `{"text": ["This field may not be blank."]}`

The client asked for 9999. The server stored 0.

Your starter repo

```
questions-api/
  config/
    settings.py
    urls.py
  questions/
    admin.py
    models.py
    serializers.py
    views.py
  manage.py
  pyproject.toml
  README.md
```

Checkpoint branches

`checkpoint-0-setup`	where you are now
`checkpoint-1-crud`	after lesson 1
`checkpoint-2-actions`	after lesson 2
`checkpoint-3-auth`	after lesson 3
`checkpoint-4-m2m`	the finished API

Fell behind?

```
git stash push -u -m "my work"
git switch checkpoint-1-crud
uv run manage.py migrate
```

Lesson 1: CRUD

Store questions and expose them over HTTP.

Models become tables

Django turns a model into a table. Each field becomes a column.

	models.py		questions_question	
text	CharField	→	varchar(300)	up to 300 characters
votes	IntegerField	→	integer	starts at 0
answered	BooleanField	→	bool	starts false
created_at	DateTimeField	→	datetime	set when it's created
id	BigAutoField	→	integer	added by Django: 1, 2, 3, ...

`makemigrations` writes the change down. `migrate` applies it.



Serializers translate and check

A serializer turns JSON into Python and back, and checks what comes in.

`POST` `/api/questions/` `{"text": "hi", "votes": 9999}`
`201 Created` `{"id": 2, "text": "hi", "votes": 0, …}`
votes is read-only, so the server ignores it.

`POST` `/api/questions/` `{"text": ""}`
`400 Bad Request` `{"text": ["This field may not be blank."]}`
Blank text fails the check. Nothing is saved.

Never trust the client.



One ViewSet, six routes

A ViewSet is one class that handles every operation on a resource. Ours is `QuestionViewSet`.

A `DefaultRouter` reads it and makes six routes:

GET	/api/questions/	list
POST	/api/questions/	create
GET	/api/questions/1/	retrieve
PUT	/api/questions/1/	update
PATCH	/api/questions/1/	partial_update
DELETE	/api/questions/1/	destroy

Who may call them? For now, anyone: `AllowAny`. Lesson 3 changes that.



What’s still broken?

Works	Create, list, read, edit, and delete questions.
Broken	`votes` and `answered` are read-only. Nothing can change them.

How should voting work?

Why not just PATCH the votes?

`PATCH` `/api/questions/1/` `{"votes": 9999}`

If votes were writable, any client could pick any number.

`POST` `/api/questions/1/upvote/` no body

The server owns the rule: one upvote, plus one.

Some operations aren't create, read, update, or delete. DRF calls them **actions**.

`detail=True` : the action works on one question, so its URL has the id in it.

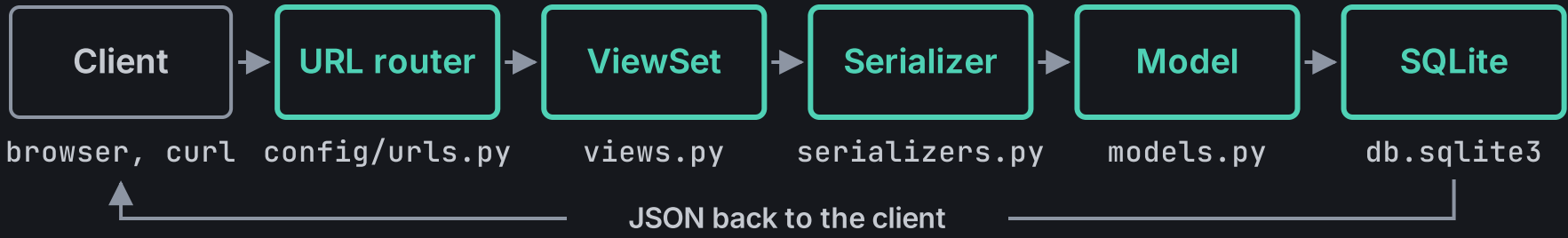
Your turn (optional)

You wrote `upvote`. Now write `toggle_answered`.

2:00

Hint: start from a copy of `upvote`. Python's `not` flips a boolean.

You built a REST API



Behind? Catch up to the end of lesson 2

```
git stash push -u -m "my work"
git switch checkpoint-2-actions
uv run manage.py migrate
```

What’s still broken?

- Anyone can upvote, forever.
- Anyone can mark a question answered.
- Anyone can edit or delete any question.

Who are you, and what can you do?

Authentication: who are you? Logging in answers it.

Authorization: what are you allowed to do? Permissions answer it.

Tonight we log in to the browsable API with a session. Apps and frontends usually send a token instead. The permission code stays the same.

The rules

Operation	Who can do it
List, read, or create questions	Anyone, including logged-out visitors
Upvote	Any logged-in user
Edit or delete questions	Staff
Toggle answered	Staff

Staff is a checkbox on each user. DRF's `IsAdminUser` checks it, not superuser. `createsuperuser` ticks both.

How DRF decides

A request reaches `QuestionViewSet`

update, partial_update, or destroy?
`get_permissions()`

yes → `IsAdminUser` staff only

no

an @action with permission_classes?
`@action(permission_classes=…)`

yes → `upvote` **`IsAuthenticated`**
`toggle_answered` **`IsAdminUser`**

no

anything else: list, create, retrieve
`permission_classes = [AllowAny]`

→ `AllowAny` anyone



What's still broken?

`sam` upvotes three times:

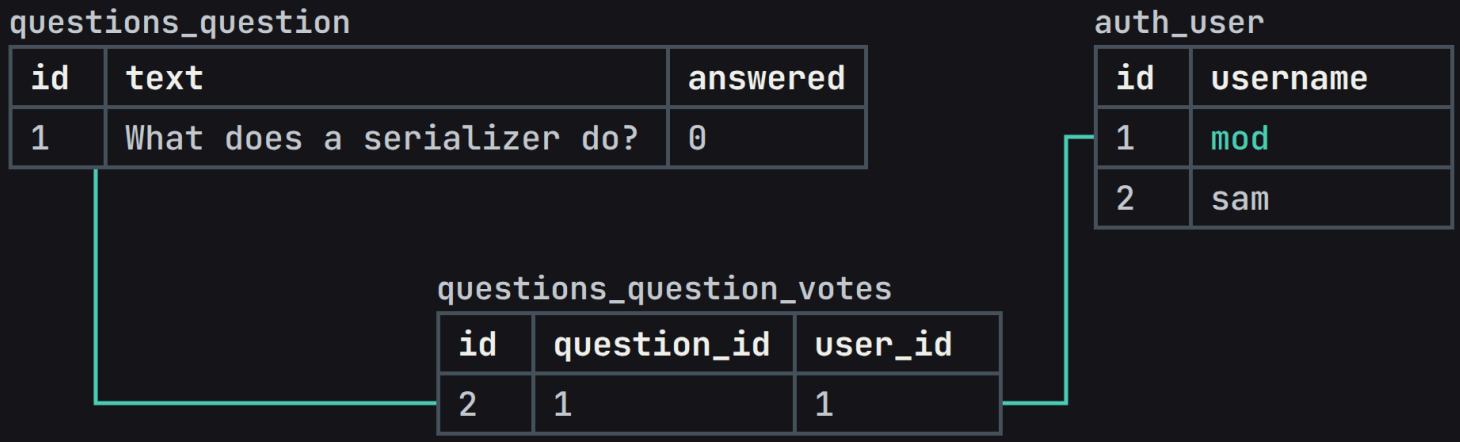
```
POST  /api/questions/1/upvote/
200 OK  "votes": 1

POST  /api/questions/1/upvote/
200 OK  "votes": 2

POST  /api/questions/1/upvote/
200 OK  "votes": 3
```

A counter can't remember who voted.

From a count to a list of voters



The serializer counts the rows

```
votes = serializers.IntegerField(source="votes.count", read_only=True)
```

```
{"id": 1, "text": "What does a serializer do?", "votes": 1, …}
```

The API doesn’t change

Clients only see the JSON. If it stays the same, they never notice.

Before: an integer column

```
{"id": 1,
 "text": "What does a serializer do?",
 "votes": 1,
 "answered": false, …}
```

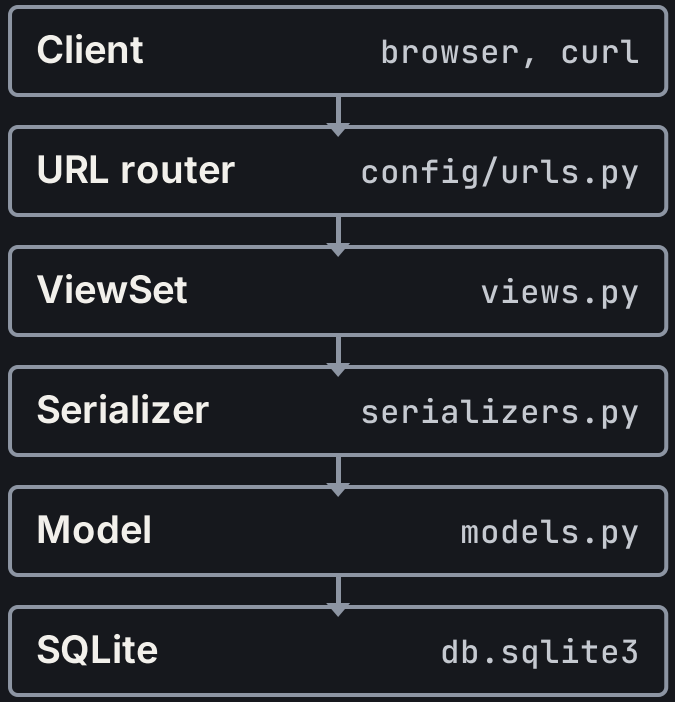
After: a count of the voters

```
{"id": 1,
 "text": "What does a serializer do?",
 "votes": 1,
 "answered": false, …}
```

One serializer field does it: `votes` reports `votes.count`.

One thing clients do notice: `/upvote/` becomes `/toggle_vote/`. Anything still calling `/upvote/` breaks.

What you built



Where to go next

- Token auth, for frontends and apps
- PostgreSQL instead of SQLite
- Tests with DRF's `APITestCase`
- Pagination and ordering
- Deploying it somewhere public
- A frontend that calls your API
- The official DRF tutorial

Placeholder: QR code. Set the URL in workshop.config.ts

Questions?

Thanks for coming. The finished code is on `checkpoint-4-m2m`.

Next Developer Club event

Placeholder: next event (set nextEvent in workshop.config.ts)

Placeholder: Developer Club logo (set logo in workshop.config.ts)



github.com/
joshualent/
questions-api

Appendix

33 Switching checkpoints

34 Troubleshooting

35 Still: one backend, many clients

36 Still: life of a POST

37 Still: from a count to a list of voters

38 Projector check

Switching checkpoints

Save your work, switch, migrate

```
git stash push -u -m "my work"
git switch checkpoint-3-auth
uv run manage.py migrate
```

Going back from checkpoint-4-m2m? Undo its migration first, while still on it

```
uv run manage.py migrate questions 0001
git switch checkpoint-3-auth
```

Or start fresh on any checkpoint: stop the server, delete `db.sqlite3`, run `migrate`.

Troubleshooting

That port is already in use

```
uv run manage.py runserver 8001
```

no such table

```
uv run manage.py migrate
```

uv: command not found (macOS, Linux)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

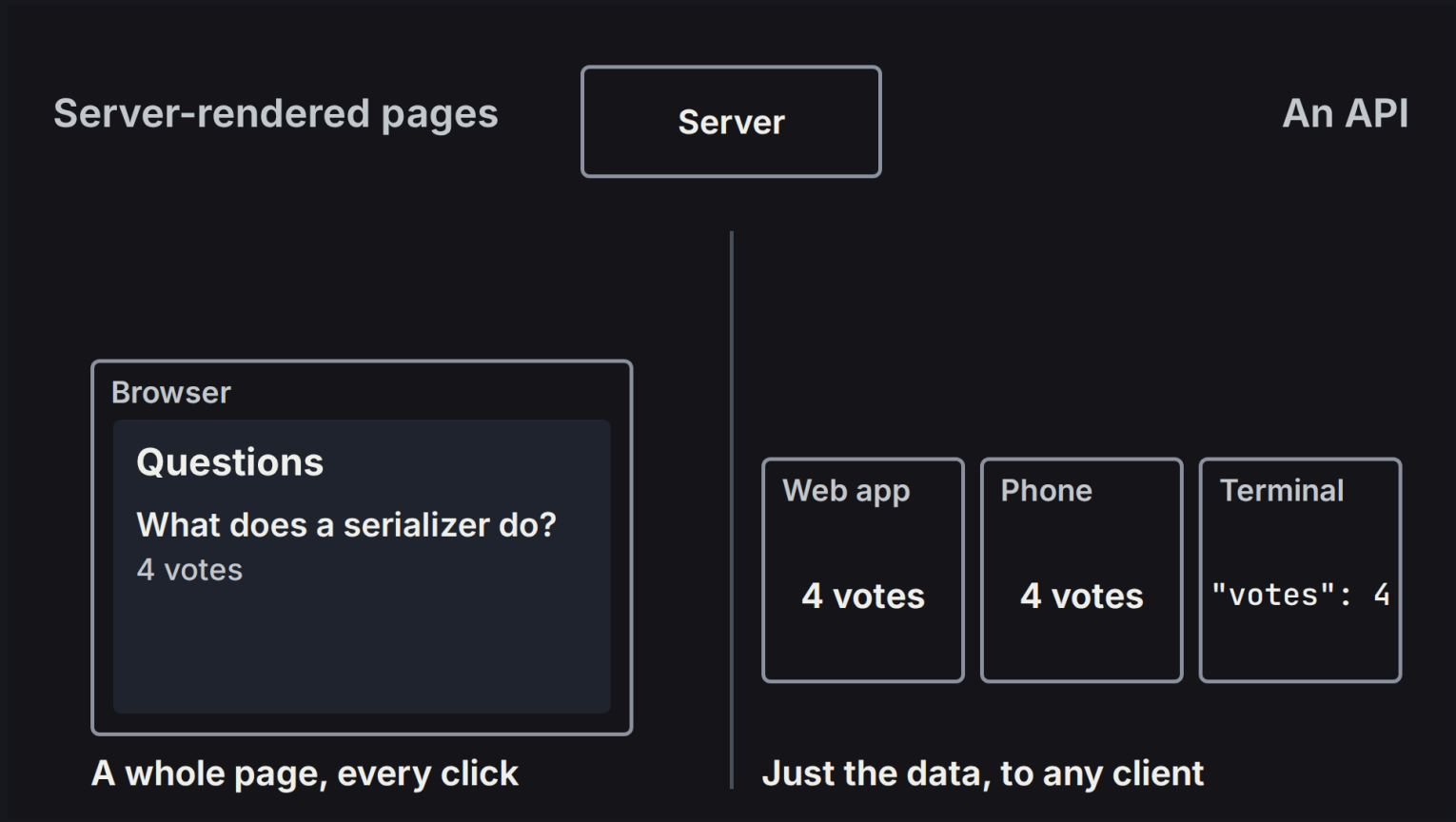
uv: command not found (Windows PowerShell)

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

No module named django

```
uv run manage.py …, not python manage.py …
```

Still: one backend, many clients



Still: life of a POST



What went in, and what came back

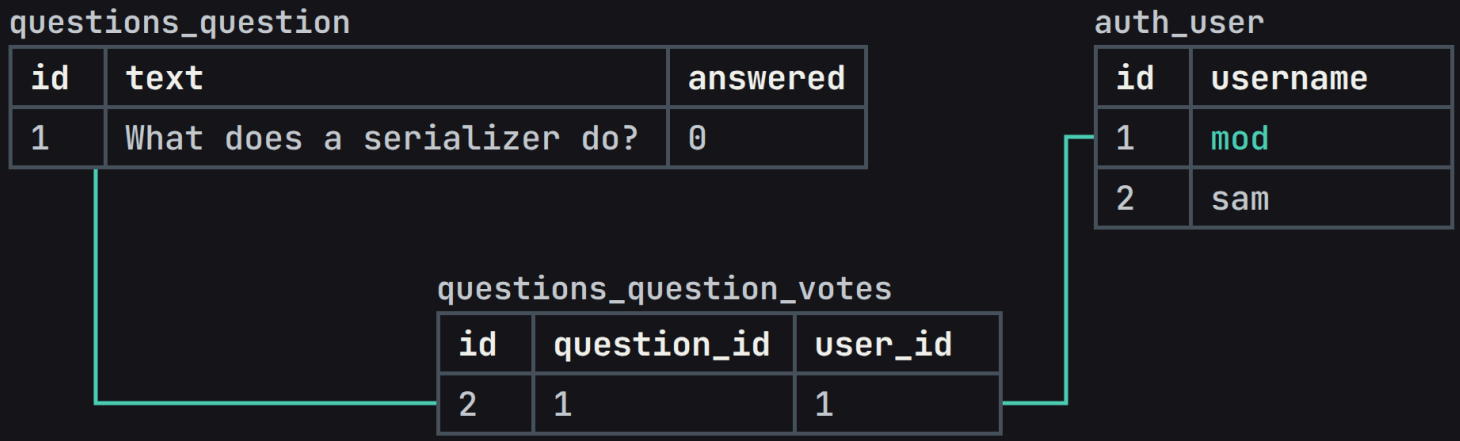
Sent `{"text": "What does a serializer do?", "votes": 9999}`

Got back `201 Created` `{"id": 1, …, "votes": 0, …}`

Blank text `400 Bad Request` `{"text": ["This field may not be blank."]}`

The client asked for 9999. The server stored 0.

Still: from a count to a list of voters



The serializer counts the rows

```
votes = serializers.IntegerField(source="votes.count", read_only=True)
```

```
{"id": 1, "text": "What does a serializer do?", "votes": 1, …}
```

Projector check

- background
- code surface
- text
- secondary text
- diagram lines
- table rules
- accent
- GET
- POST
- PUT / PATCH
- DELETE
- 2xx
- 4xx

Body text, 24 px: the smallest body size in the deck.

Secondary text, 24 px: still readable at the back?

```
Code, 18 px: question.votes.add(request.user)
```

Diagram label, 16 px: config/urls.py · serializers.py · db.sqlite3

GET POST PATCH DELETE 201 Created 403 Forbidden

```
class Question(models.Model):
    text = models.CharField(max_length=300)
    votes = models.IntegerField(default=0)
```